

1. What is Multithreading?

Multithreading allows a program to execute **multiple tasks (threads)** concurrently within a single process.

👉 In simple terms:

Instead of doing one task at a time, your application can do many tasks **simultaneously or in parallel**.

Multithreading is the ability of a process to execute **multiple threads of execution concurrently**.

- A **process** = running application
- A **thread** = execution path within that process

👉 Key idea:

Multiple threads share the **same memory space**, but execute **independently**.

🔍 Real-Life Example

Think of a food delivery app:

- One thread → Taking orders
- Another thread → Assigning delivery partner
- Another thread → Tracking delivery

All happen **at the same time**, not sequentially.

🔍 Why Multithreading?

- Improve responsiveness (UI doesn't freeze)
 - Utilize CPU efficiently
 - Perform parallel operations
-

⚙️ 2. What is a Thread in C#?

A **Thread** is the smallest unit of execution.

In C#:

- Every program starts with a **Main Thread**
 - You can create additional threads to perform background work
-

🔧 3. Creating and Starting a Thread

```
using System;
```

```
using System.Threading;
```

```
class Program
```

```
{
```

```
    static void Work()
```

```
    {
```

```
        Console.WriteLine("Thread started");
```

```
    }
```

```
    static void Main()
```

```
    {
```

```
        Thread t = new Thread(Work);
```

```
        t.Start(); // Starts execution
```

```
    }
```

```
}
```

⚙️ 4. Thread Class – Important Properties

◆ 1. IsAlive

Checks if thread is still running.

```
Console.WriteLine(t.IsAlive);
```

◆ 2. Name

Assign a name to thread (useful for debugging)

```
t.Name = "WorkerThread";
```

◆ 3. Priority

Controls execution priority

```
t.Priority = ThreadPriority.Highest;
```

Values:

- Lowest
 - BelowNormal
 - Normal (default)
 - AboveNormal
 - Highest
-

◆ 4. IsBackground

```
t.IsBackground = true;
```

- False (default) → Foreground thread
 - True → Background thread (terminates when main thread exits)
-

◆ 5. ThreadState

```
Console.WriteLine(t.ThreadState);
```

Thread Lifecycle

A thread goes through different states:

1. Unstarted → Created but not started
2. Runnable → Ready to run
3. Running → Currently executing
4. Wait/Sleep/Blocked → Waiting for resource/time

5. Stopped (Dead) → Execution completed

Possible states:

- Running
 - WaitSleepJoin
 - Suspended (obsolete)
 - Stopped
-

5. Thread Class – Important Methods

1. Start()

Starts thread execution.

```
t.Start();
```

2. Sleep(int milliseconds)

Pauses current thread.

```
Thread.Sleep(1000); // 1 second
```

 Moves thread to Wait/Sleep state

3. Join()

Main thread waits until another thread completes.

```
t.Join();  
Console.WriteLine("Thread finished");
```

4. Interrupt()

Interrupts a thread that is in Sleep/Wait.

`t.Interrupt();`

👉 Throws `ThreadInterruptedException`

✅ 5. `Abort()` ⚠️ (Obsolete)

`t.Abort();`

- Terminates thread immediately
 - Unsafe → NOT recommended
-

✅ 6. `Suspend()` / `Resume()` ⚠️ (Obsolete)

- Pauses/resumes thread
 - Causes deadlocks → Not used in modern C#
-

✅ 7. `CurrentThread`

Gets currently executing thread

`Thread current = Thread.CurrentThread;`

`Console.WriteLine(current.ManagedThreadId);`

🔄 4. Foreground vs Background Threads

Type	Behavior
Foreground Thread	Keeps app alive
Background Thread	Stops when main thread ends

`Thread t = new Thread(PrintNumbers);`

`t.IsBackground = true;`

`t.Start();`

Problem in Multithreading – Race Condition

When multiple threads access shared resource → inconsistent results

```
int counter = 0;
```

```
void Increment()  
{  
    for (int i = 0; i < 1000; i++)  
    {  
        counter++;  
    }  
}
```

👉 Expected: 2000

👉 Actual: unpredictable

🔒 7. Synchronization (Core Concept)

Synchronization ensures that:

Only one thread accesses a critical section at a time

🔒 8. Critical Section

A critical section is:

Code that accesses shared resources (memory, file, DB, etc.)

🔧 9. Synchronization Techniques

✅ 1. lock (Most Common)

```
object obj = new object();
```

```
lock (obj)  
{  
    counter++;  
}
```

👉 Internally uses Monitor

✓ 2. Monitor

More control than lock

```
object obj = new object();
```

```
Monitor.Enter(obj);
```

```
try
```

```
{
```

```
    counter++;
```

```
}
```

```
finally
```

```
{
```

```
    Monitor.Exit(obj);
```

```
}
```

◆ Monitor.Wait() and Monitor.Pulse()

Used for thread communication

```
Monitor.Wait(obj); // releases lock and waits
```

```
Monitor.Pulse(obj); // signals waiting thread
```

10. Deadlock

A deadlock occurs when:

- Thread A waits for Thread B
- Thread B waits for Thread A

👉 Both stuck forever

✗ Example

```
lock (obj1)
```

```
{
```

```
    lock (obj2)
```

```
{  
}  
}
```

```
lock (obj2)  
{  
    lock (obj1)  
    {  
    }  
}
```